Timestamp

Andrej Karpathy Lecture 1 - Intro to Neural Network & Backpropagation 15 Sep 26.

0 → Micrograd [Implement Backpropagation]

- Wrap numbers to value objects
- Build expression graphs by addition, subtraction, mul, div, power etc.
- If value objects 'a' and 'b' gives 'c' i.e c = a+b, then it keeps pointers. i.e 'c' keeps pointers to 'a' and 'b' as its child nodes.
- If a large expression gives 'g' as output value, the output of the forward pass can be evaluated as g.data
- g.backward() initialize backpropagation at the node g. This evaluates derivative of 'g' wrt all the internal nodes. It evaluates the derivative of 'g' wrt nodes 'c', 'd', 'e', 'f' and also inputs 'a' & 'b'.
Derivative of 'g' wrt 'a' is a.grad. Similarly for others.
- The derivative gives the information of how 'a', 'b', 'c'... are affecting 'g' through this mathematical expression
- If b.grad i.e $\frac{dg}{db}$ is 645.576, if we slightly increase 'b' the 'g' will grow and slope of this growth is 645.576. This says how 'g' will respond if 'b' gets tweaked by a tiny amount.
- Neural networks as a special class of mathematical expressions. Whereas backpropagation is more general and also applies to a whole other lot of mathematical expressions.
- In production, we have n dimensional tensors in place of the single number wrapped in value object, that we used here in micrograd. The numbers as tensor, take advantage of parallelism in computers to do efficient processing. But none of the math changes.
- Micrograd is everything that is required to train neural networks and everything else is just efficiency. i.e autograd engine
- Entire micrograd implementation is in 'engine.py'. Neural network implementation is in 'nn.py'.

8.31 → Intuitively understand Derivatives

$f(x) = 3x^2 - 4x + 5$. For a set of inputs x from -5 to 5 in steps of 0.25, plot this function.

We could take derivative of the function using calculus. But for complex expressions like neural network, this will be difficult.

Definition of what is differentiable $L = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$

If we nudge the value of function at point x by a small amount h, what is the sensitivity with which the function respond. Will it go up or down. What is the slope? Find derivative at a point by taking a small value of h. Finds how much the function responded $f(x+h) - f(x)$ over the run (h).

For an expression $d = a*b+c$, we can find derivative of 'd' w.r.t 'a', 'b' & 'c' by taking the above definition of differentiable.

→ Value object in Micrograd. 19.20

Why wrap a single number into value object?.

For a standard number like $a = 2.0$, by wrapping it inside a Value object, you turn a passive number into a smart node in a computation graph.

Raw number : 2.0

Value object : Has data → 2.0

grad → 0.0

_prev → (a, b) to track operations. Stores data w.r.t immediate previous values that created instant value.

Support operations like add, multiply (overloading)

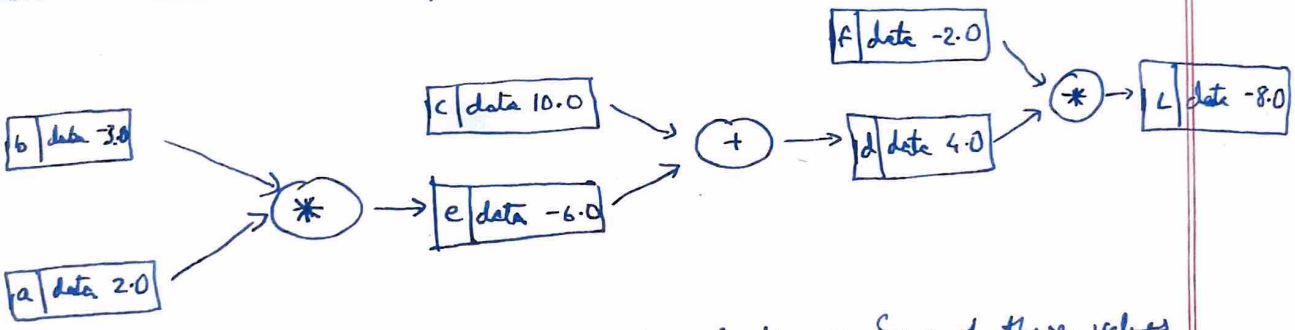
→ graphviz library is used to draw the graph. 25.52

Used the code, as is.

→ Forward Pass. {what we did until 29.34 minutes}

We took scalar values of a, b, c, e, f and build mathematical expression using '+' and '*' operations till now. We got the output value L.

This process is called forward pass



In neural network this L represent the loss function. Some of these values a, b, c etc will represent the weights. In backpropagation we need to find derivative of L wrt each of the node i.e $\frac{dL}{dL} = 1$, $\frac{dL}{df}$, $\frac{dL}{dd}$, $\frac{dL}{df}$, 30.29

$\frac{dL}{dc}$, $\frac{dL}{de}$, $\frac{dL}{db}$, $\frac{dL}{da}$.

We generally dont use the derivative of loss wrt data (leaf nodes), because this input data is fixed. So only weights are used for tuning

Now create a variable .grad inside value class for storing the derivative

The grad is initialized to 0, i.e, we assume that changing this value will not change the output at initialization

$\frac{dL}{dL} = 1.0$ is the gradient of node L.

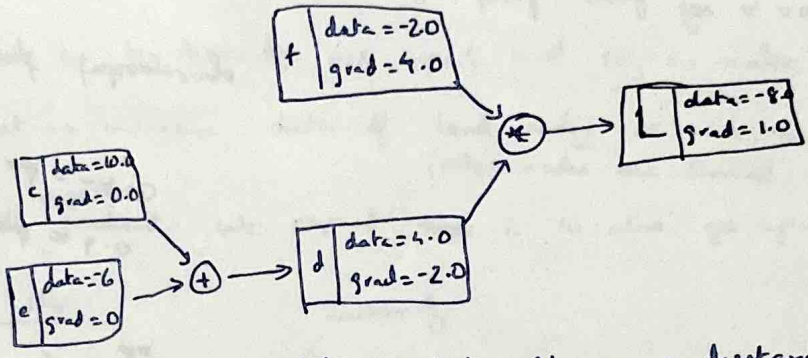
$f * d = L$

$L = f * d$

$\frac{dL}{df} = d$ i.e d.data

$\frac{dL}{dd} = f$ i.e f.data.

} i.e gradient of a node while multiplying two nodes is the data value of the other node.



Now coming to nodes 'c' and 'e'. If you understand how to calculate gradient of output node L wrt intermediate node c, then you understand whole of gradient calculation. We know how nearby node d affects L. Now we need to find how a distant node c affects L.

By chain rule in calculus

$$\frac{dL}{de} = \frac{dL}{dd} \times \frac{dd}{dc}$$

wrt next node

It is simply the product of local gradient $\frac{dd}{dc}$ and gradient of output node wrt the next node

$d = c + e$

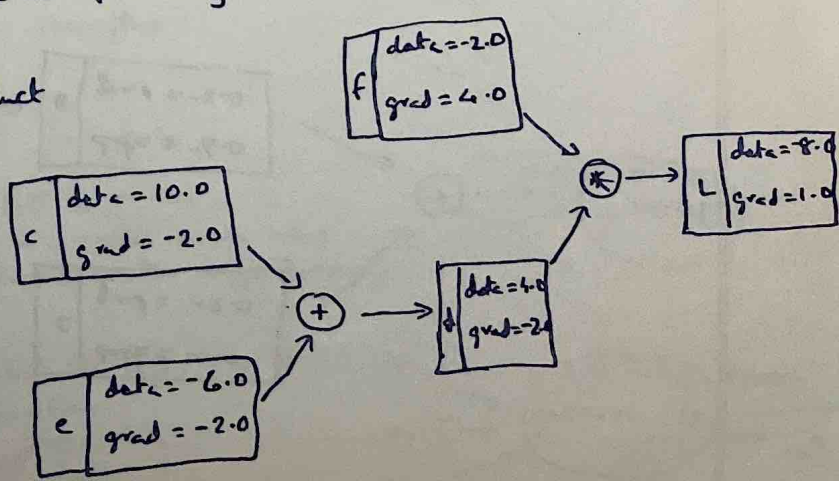
$\frac{dd}{dc} = 1.0$

} i.e addition at a node will simply transfer the gradient from previous node to present node, without any change. The gradient just flow to child node

$\frac{dL}{dd} = f = -2.0$, as obtained previously.

So, $\frac{dL}{dc}$ is the product

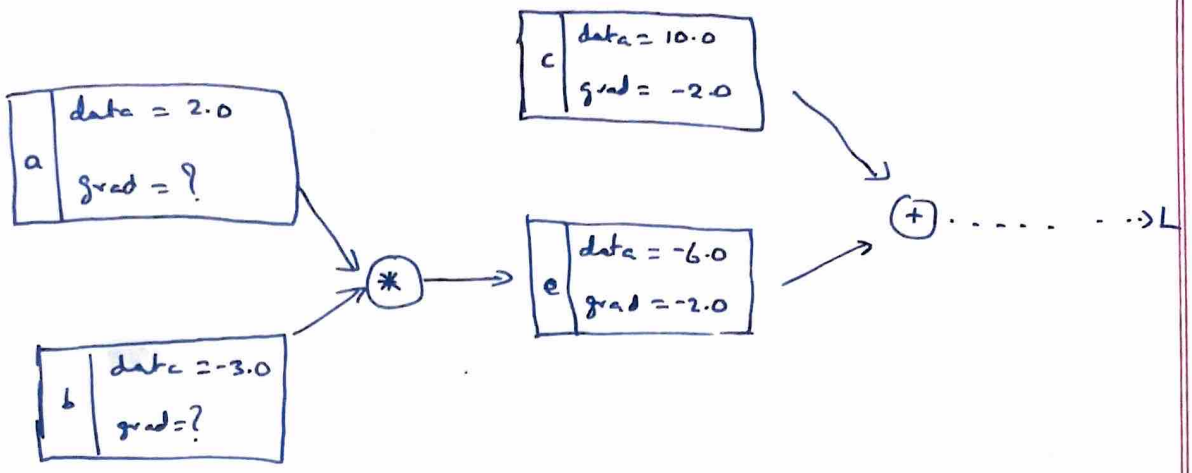
$$\frac{dL}{dc} = -2.0 \times 1.0$$
$$= -2.0$$



Ily, by symmetry

$$\frac{dL}{de} = -2.0 \times 1.0$$
$$= -2.0$$

49.0



Now for a multiplication node somewhere in the graph, consider local derivative,

for $a \times b \to e$, the local derivatives $\frac{de}{db} = a.data$ & $\frac{de}{da} = b.data$

as found out earlier. i.e cross of data

$$\frac{de}{db} = 2.0 \qquad \frac{de}{da} = -3.0$$

Now by chain rule

$$\frac{dL}{db} = \frac{dL}{de} \times \frac{de}{db} = e.grad \times \frac{de}{db} = -2.0 \times 2.0 = -4.0$$

$$\text{//y} \quad \frac{dL}{da} = \cancel{xxx} \; -2.0 \times -3.0 = 6.0$$

$a.grad = 6.0$

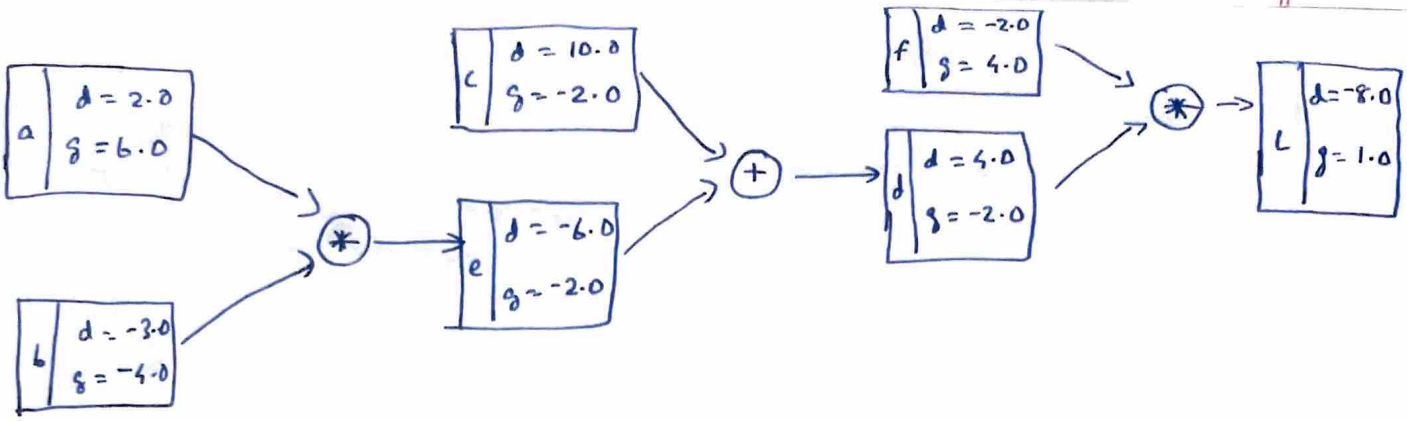
$b.grad = -4.0$

Two important takeaways

1. For multiplication, the local grad for a node is data value of other node.
2. For addition, the local grad is just same as grad of output.

By chain rule, find grad wrt out using local grad multiplied by grad value of output node.

Backpropagation is recursive application of chain rule, backwards through the computational graph.



51.10

Now let us try to nudge our inputs and try to make L go up.

So let us increase data of lead nodes ie a, b, c and f by a fraction of its respective gradient. We expect ~~toss~~ L to also go up as we do this.
(Other nodes are formed from these)

One step

fraction of
$\downarrow$
$a.data = a.data + 0.01 \times a.grad$

$b.data = b.data + 0.01 \times b.grad$

$c.data = c.data + 0.01 \times c.grad$

$f.data = f.data + 0.01 \times f.grad$

now do forward pass

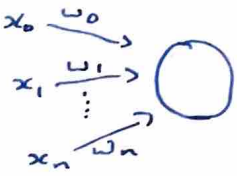
$e = a \times b$

$d = c + e$

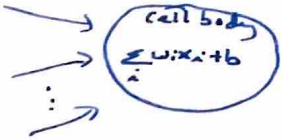
$L = f \times d$

This gives $L = -7.28$. So we got a way to influence output by tweaking inputs in a complex expression graph.

Neuron – ① Inputs multiplied by weights and added (Synapses)
weights are synaptic strengths.



② Add a bias to this i.e + b
(Inbuilt trigger of neuron.
Tweaking this makes neuron more trigger happies or less trigger happies)



③ Apply activation function to this i.e f()
(This is a squashing function.
eg. tanh)



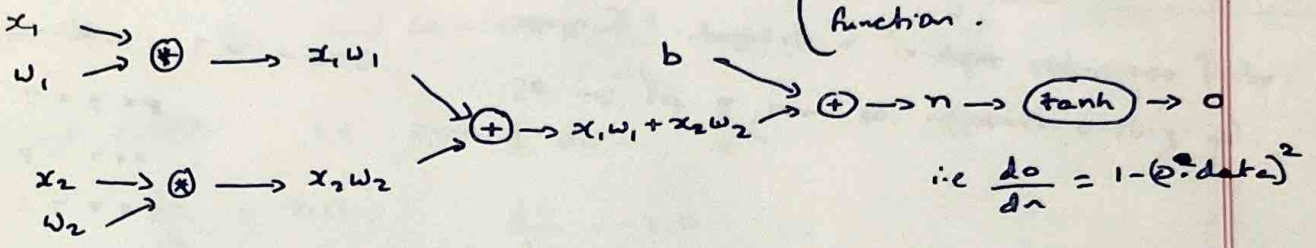
Presently we have only addition and multiplication supported for value object- for activation function, we need to implement tanh. When we implement a new operation for value object, the only criteria is that we should be able to do local derivative for that operation, to do backpropagation.

$$\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}$$

As long as we know the derivative of tanh, we will be able to backpropagate through it.

$$\frac{d\tanh x}{dx} = 1-\tanh^2 x = 1-(\tanh x)^2$$

$\tanh^2(x)$ is simply a mathematical shorthand for squaring the entire output of the hyperbolic tangent function.



This is a single neuron. In a neural network, there are many such connected neurons. We are interested in optimizing the weight values of neurons to minimize the loss function of the network.

Implementing code to automatically do backpropagation starts here. → 1:09.28

In Value object store a special '_backward' variable.
This will be a lambda function that does the little piece of chain rule for a node. In each node, we are going to store how the output gradient is chained to the input gradients. The backward function in value object doesn't do anything, by default

i.e `self._backward = lambda: None`

This will be the only case for a leaf node, i.e nothing to do.

Now when we do an operation in value object, set the function to be one that propagates the gradient.

Inside the definition of an operation (like addition), we define the `_backward()` function. Then we write expression for self.grad and other.grad i.e children node grads, from parent node grad i.e out.grad.

After defining the _backward() function for an operation, we ~~don't call~~ the function inside the operation, just below the function definition for the node, wrt its output.

Now this function is ~~automatically executed~~ defined when we do the operation.

Two things we did till now in code to automate backpropagation :-

① Within __init__ of Value class added

self._backward = lambda : None // function definition.

② within operation (like +) function, defined and ~~called~~ pointed to this _backward definition

```
def __add__ (self, other):
    out = Value(self.data + other.data, (self, other), '+')
    def _backward():
        self.grad = 1.0 * out.grad
        other.grad = 1.0 * out.grad
    out._backward = _backward
```

// Here the new value object "out" is created. The data as well as grad value of this new object is to be found. For data, it is output of operation calculated. For grad, it is output of _backward function as defined here, but not calculated in forward pass.

// This is chain rule. 1.0 since local derivative of '+' is 1.0.

// When the addition operation is done, now the _backward function is defined and pointed for the 'out' value created. So here in the function both out Value is created i.e data calculated and how the grad will get propagated from out to self and other is defined. Only after complete forward pass of data creation, we will backpropagate. But definition exists now wrt how to backpropagate. Order of calling backpropagation is hence important.

Note: We are not calling the function like _backward(). But we are just pointing to it out._backward = _backward. If we call it here, this always makes it NoneType. Error at lecture ~~video~~ video 1:15.40

1:15:40

Calling n._backward() will now populate gradient values for children from the parent node. Here we have to populate gradient values from output to inputs in expression graph, in that sequence.

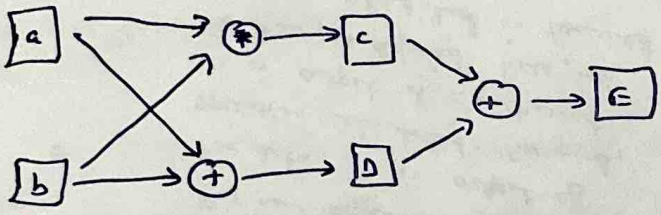
For automating the _backward call for the complete graph, we need to sort the nodes. Use Topological sort.

```
topo = [ ]                          // It maintains a list
visited = set ( )                   // Set of visited nodes, no duplicates
def build_topo(v):
    if v not in visited:
        visited.add(v)
        for child in v._prev:
            build_topo(child)
        topo.append(v)
build_topo(o)
topo
```

No need to learn this code.
But understand this :-
This function adds a node to the list, only after all its children are added in the list.

In reversed order of nodes in topo list, call the _backward() function. 1:23.0

One Error



Here grad of a and b gets calculated through two paths. First from grad of c and multiplication, then through grad of D and addition. ~~So both grad gets overwritten~~. First grad will be overwritten when the second grad is calculated.

In next cycle, grad will again get initialized to 0

To solve this, keep accumulating the grad in a backpropagation cycle.
Do this inside ~~-backpropagation~~ backward function definition within operations.

Inside add

```
self._backward += 1.0 * out.grad   etc.
```

Now let us practice implementing some more operations like division, subtraction etc.

First let us implement adding of a constant number (int) to a value object.

If we try a + 2, error will be that the int 2 has no attribute 'data'.

check and make the int a value by

```
other = other if isinstance(other, Value) else Value(other)
```

If other is a Value object

wrap the number as value object.

__rmul__ in python.

```
def __rmul__(self, other):
    return self * other
```

First the multiplication of self & other (self * other) is attempted in __mul__. If it fails due to some reason, the python will attempt multiplication in reverse order i.e other * self, if we define this function.

with 'math.exp(x)'

Though we have implemented tanh x and its differentiation, let us attempt the same using elementary implementation of exponentiation and division operations.

1:30.37

```
def exp(self):
    out = Value(math.exp(self.data), (self,), 'exp')
    def _backward():
        self.grad += out.data * out.grad
                     (de^x/dx = e^x)
    out._backward = _backward
    return out
```

Exponentiation

For division operation, let us do something more general.

$a^b = a \times (b)^{-1}$. Here we already have multiplication. We only need to define a power function. & do the power -1 is required here.

```
def __truediv__(self, other):
    return self * other**-1
```

division

```
def __pow__(self, other):
    assert isinstance(other, (int, float)), "only support int/float now"
    out = Value(self.data ** other, (self,), f'**{other}')
    def _backward():
        self.grad += (other * (self.data)**(other-1)) * out.grad
    out._backward = _backward
    return out
```

Power

```
def __neg__(self):
    return self * -1
def __sub__(self, other):
    return self + (-other)
```

subtraction. with existing definitions of multiplication & addition.

1:36.25

By implementing tanh as its elementary composite operations using exponentiation, addition, subtraction and division (Cell number 32 in Jupyter notebook), we get two things.

$$\tanh x = \frac{e^{2x}-1}{e^{2x}+1}$$

1. The output data from forward pass for the expression is same as that using tanh.
2. The gradient value for input nodes after backpropagation is same as that when tanh was used.

Further in the graphviz representation, instead of a simple tanh node, it shows a new and more number of nodes.

Implementing the same functionality as above, using a modern deep neural network library 'PyTorch'. Micrograd is roughly modeled around this library. Micrograd only work on scalars. But in pytorch everything is 1:39.20

Tensor = 'n' dimensional 'array' of 'scalars'.

↳ like 2x3 dimension array, so it is a matrix

Micrograd	PyTorch
Value object	Tensor

```
import torch   // imports PyTorch
torch.Tensor([[1,2,3],[4,5,6]])   // This gives output tensor([[1.,2.,3.],
                                                            [4.,5.,6.]])
```

(Capital 'T')

Data type of tensors is 'float32' by default.
Python by default uses 'double' i.e 'float64' point precision.
Further Python does not have float32 precision implementation.
So we cast all scalars in tensor to double.

float32 has 32 bits per number. It represents 7 decimal digits of accuracy.
double/float64 has 64 bits per number. It represents 15 to 16 decimal digits of accuracy.

PyTorch by default assumes that the nodes that are getting defined does not need gradients. So we need to explicitly say if a node needs gradient. It is set to false by default due to efficiency reasons as the leaf nodes does not need gradients.

The tensor objects in micrograd has data attribute and grad attribute. Only difference is that we have to call a '.item()' on the data and grad to get the element from the tensor. It takes single tensor of one element and returns the element by stripping out the tensor wood over it.

The tensor object also has a '.backward()' function. 1:43.0

0.item() and 0.data.item() produces the same result in PyTorch.

1:43.56

Building a 2 layered ~~Multilayer~~ Multilayer Perceptron.

First implementing a single neuron in PyTorch.

Above we have watched autograd in micrograd to implementation in PyTorch.
Now moving on to the neural network side.

Single Neuron implementation

```
class Neuron:                                   # number of inputs
    def __init__(self, nin):                    # random numbers, as per number of inputs
                                                # converted to value object.
        self.w = [Value(random.uniform(-1,1)) for _ in range(nin)]   # w is initialized.
        self.b = Value(random.uniform(-1,1))

    def __call__(self, x):                      # sum takes a second argument to start adding from.
        act = sum((wi*xi for wi,xi in zip(self.w, x)), self.b)
        # act: activation value i.e (wi*xi + b)
        # zip pairs the two iterators passed to it like this
        out = act.tanh()
        return out
```

$$\text{zip}\left(\begin{bmatrix}1\\2\\3\end{bmatrix}, \begin{bmatrix}4\\5\\6\end{bmatrix}\right) = \begin{bmatrix}(1,4)\\(2,5)\\(3,6)\end{bmatrix}$$

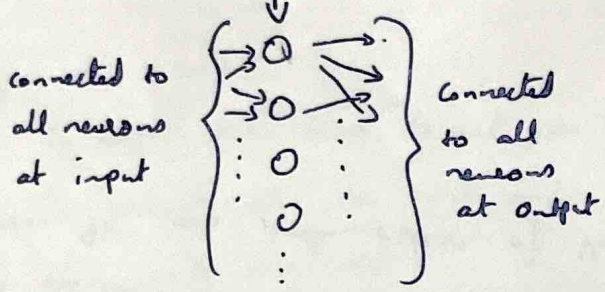
input — weight

```
x = [2.0, 3.0]      # inputs
n = Neuron(2)       # with 2 inputs
n(x)                # using __call__ dunder for the class object.
```

1:48.00 Will the above code work with just 'import torch' and without an implementation of Value class?

No. It needs implementation of Value class of autograd to work.

Layer of neurons



In a layer, the neurons are not connected to each other.

So layer is a list of neurons. We take the number of neurons in a layer as the input argument. This is number of outputs in this layer and taken as 'nout' of variable for the layer. Here each neuron in the layer has 'nin' dimensionality as before. Now instead of Neuron, we can have Layer of them with number of inputs ('nin') i.e dimension for each neuron wrt number of weights and number of outputs ('nout') i.e number of neurons in the layer as arguments to Layer class.

If each neuron has 2 dimensions, and there are 3 neurons in a layer,

```
class Layer:

    def __init__(self, nin, nout):
        self.neurons = [Neuron(nin) for _ in range(nout)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs

x = [2.0, 3.0]
n = Layer(2,3)
n(x)
```

← List of neurons. Total nout neurons. Each with dimension nin. This initializes w & b in Neuron class

← for each neuron, call __call__() function of Neuron class. This calculates the neuron output. Keep it as a list.

← inputs

← Layer class initialized with nin=2 & nout=3 (nin: dimension i.e weights, inputs; nout: number of neurons. i.e outputs)

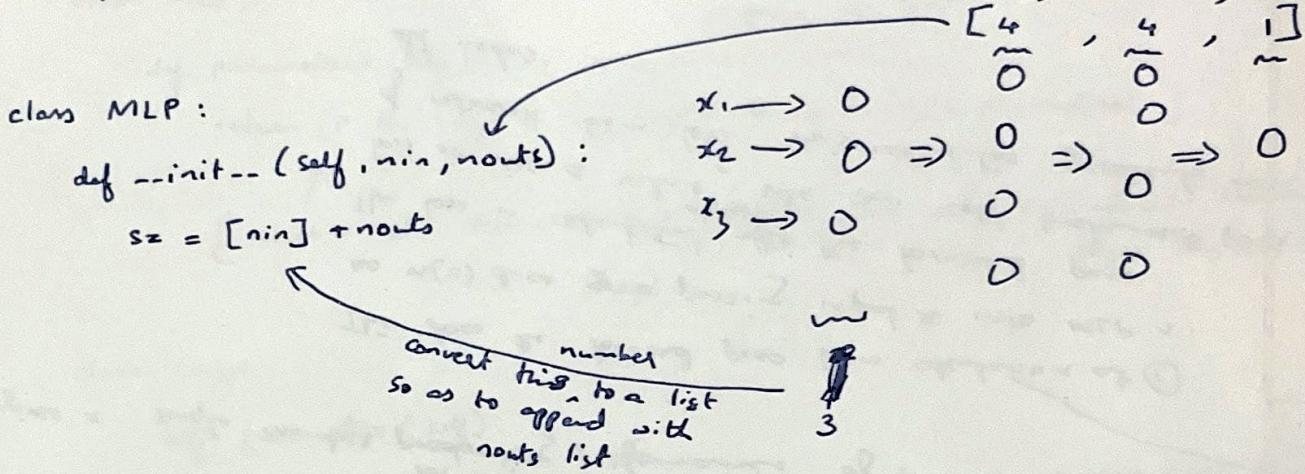
← calling each neuron in layer with input x

MLP (Multi Layer Perceptron)

Inside `__init__()`,

The layers feed into each layer sequentially. Number of neurons in each layer of MLP is given is as a list. Iterate over adjacent pair of layers across in MLP over entire number of layers. Inside `__call__()`, for MLP just pass the input i.e 'x' for passing down to Layer class and down to Neuron class.

'⇒' means fully connected




```
class MLP:
    def __init__(self, nin, nouts):
        sz = [nin] + nouts
```

Now `sz = [3, 4, 4, 1]` in this case

```
self.layers = [Layer(sz[i], sz[i+1]) for i in range(len(nouts))]
```
(len(nouts) = No. of layers − 1)

~~or else~~ [Layer(3,4), Layer(4,4), Layer(4,1)]

```
def __call__(self, x):
    for layer in self.layers:
        x = layer(x)
    return x
```

} This just passes x from MLP to Layer class and Neuron class

```
n = MLP(3, [4, 4, 1])
x = [2.0, 3.0, -1.0]
n(x)
```

In cell 28 of Jupyter notebook, we drew this graph using graphviz library. It is complex. We will not be able to easily differentiate for this on paper and find the gradients. For this we have micrograd. So we can backpropagate to weights.

Now to train the MLP, we need a dataset.
So let us take a simple binary classified dataset and define it as follows:-

Dataset

```
xs = [ [2.0, 3.0, -1.0],
       [3.0, -1.0, 0.5],
       [0.5, 1.0, 1.0],
       [1.0, 1.0, -1.0],
     ]

ys = [1.0, -1.0, -1.0, 1.0]
```

input values x_1, x_2, x_3, x_4 gives y_1, y_2, y_3, y_4 → 1.0, -1.0, -1.0, 1.0

classified outputs

```
ypred = [n(x) for x in xs]
```

→ (1)

When we pass the xs to MLP we implemented, we get some predicted outputs. This needs to closer to our desired output. How we will tune the neural network to do this?

The trick used in deeplearning is to measure a single number (Loss) that calculates the total performance of the neural network.
When neural network is not performing good, loss will be high. We need to minimize this loss.

One way is to calculate sum of squared differences between predicted value and desired (ground truth, gt) value.

```
loss = sum((yout - ygt)**2 for ygt, yout in zip(ys, ypred))
```

This gives a single number for current predicting performance of the neural network.

1:54.55

This does the forward pass from calculation at (1) as n(x) does ~~string~~ passing input x into MLP n.
Then loss is calculated after the forward pass.
loss is also a value object: we can call backward pass of network from loss. This calculates gradients for all nodes.

```
loss.backward()
```

Now we need a function in MLP class to return the total parameters lists i.e all weights and biases of network. This will be used to change only the data value of parameters (i.e respective gradients) so that the output can be controlled.

```
class MLP:
    def parameters(self):                                   # Nesting for loops
        return [p for layer in self.layers for p in layer.parameters()]
```

{ return a list. For each layer in MLP, this calls parameters() function in layer object/class }

```
class Layer:
    def parameters(self):
        return [p for neuron in self.neurons for p in neuron.parameters()]
```

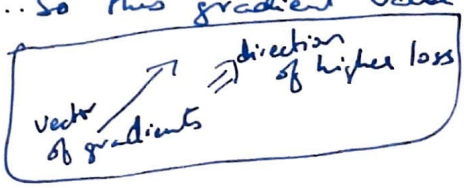
{ return a list. For each neuron in Layer, this calls parameters() function in Neuron object/class }

```
class Neuron:
    def parameters(self):
        return self.w + [self.b]
```

{ w is already a list. Convert scalar b to a list and append. This is the list of parameters }.

Now recalculate the loss of network and do backpropagation for this loss on the MLP. Now we do gradient descent.

In gradient descent, we take gradient as a vector pointing at direction of loss. i.e higher the gradient, more the loss.. So this gradient value has to be subtracted from data.



Run following steps again and again to change parameter values for lesser loss and better predictions.

1. Forward pass for predictions
2. Calculate loss
3. Backward pass
4. Updation of parameters (w & b) data with new gradient
5. Repeat

→ loss will reduce with each iteration.

Tuning the neural network. Finding the correct step size (i.e fraction to multiply gradient prior updating data value) and number of iterations gradient descent to run the updation loop for parameters. This is a skill. If you over step or run too many loops, the loss function will jump to a region farther away from the minimum.

2:14:06

~~Bonus~~ Summary!